

INVESTIGATION REPORT

Issue 198: CSR write slips through while debug halts the pipeline

openhwgroup/cv32e40x

Document	RTL-TRIAGE-CV32E40X-198
Disposition	Bug confirmed, fix verified
Record date	2026-09-18
Classification	Internal working document. Not a verification sign-off.

1 Status

Issue	openhwgroup/cv32e40x#198
Issue type	Bug report from an integrator or user
Affects	cv32e40x
Configuration	pre-fix RTL 9e309e22 (2021-08-24) vs master d952cd6; X_EXT=0
Disposition	Bug confirmed, fix verified
Severity	Not assessed
Agreement	Same disposition as the independent human record.
Evidence	6 cited, 5 present, 1 not retained

Disposition meaning. The reported behaviour was reproduced, and the checks listed under Evidence pass against the corrected design.

Severity is not recorded for this run. The harness does not infer a severity from the disposition, so the field is reported as not assessed rather than filled in.

2 Description

Blind hypothesis from the report ('mscratch updated when a debug request interrupts a CSR instruction'): the CSR write port in cv32e40x_cs_registers gates csr_we_int with a local instr_valid that, at the pre-fix revision, is ex_wb_pipe_i.instr_valid && !kill_wb only; halt_wb is not included (the RTL even carries a '// && !halt_wb; todo:ok: Not SEC equivalent' comment). When an external debug request arrives while a CSR instruction sits in WB, the controller halts WB (pending_debug && debug_allowed -> halt_wb) for the cycle before DEBUG_TAKEN, and the halted instruction still performs its CSR write; DEBUG_TAKEN then kills WB and sets dpc to that instruction, so it executes again after dret (double write, or a write that was supposed to be cancelled). Two properties were written on the write port: P1 an instruction flagged illegal never writes a CSR (never violated on either revision: the EX stage already clears csr_en for illegal CSR accesses, and the 'illegal CSR in WB' cover is unreachable on both), and P2 a CSR write happens only when WB is neither halted nor killed. P2 fails on the pre-fix RTL at BMC step 8: csrrs x0, mstatus, x1 in WB, debug_req_i rises, halt_wb=1 and csr_we_int=1 in the same cycle, DEBUG_TAKEN next. On master both properties pass unbounded (abc pdr) and BMC-30, with the CSR-in-halted-WB and mscratch-write covers reached at step 7 (not vacuous). Human fix: PR #201 'Halting CSR writes in WB' (merged 2021-08-26, d2ec2200) adds && !ctrl_fsm_i.halt_wb to exactly that instr_valid, the same predicate as P2. The report names mscratch and an illegal CSR instruction; the formal witness shows the defect is independent of the CSR address and of the illegal flag: any CSR write instruction in WB at the moment of a debug halt.

3 Conditions

Not recorded as separate fields for this run. The conditions and environment assumptions that bound this result are stated inside the Description, and the configuration is given in Status.

4 Implications

Not recorded as a separate field for this run. What the disposition means for a user of the core follows from the Disposition meaning in Status and the bounds in Limitations and scope.

5 Action

Checker used

checker_198.sv

6 Evidence

Ref	Type	Path, relative to the run directory	File	Result recorded
E1	counterexample	formal/prefix_bmc/job/engine_0/trace.vcd	not kept	pre-fix: a_198_write_only_when_retiring fails at step 8: csr_we_int=1 for mstatus (0x300) while ctrl_fsm_i.halt_wb=1 (debug_req_i=1, pending_debug), DEBUG_TAKEN at step 9 (cex_prefix_csr_write_in_halted_wb.png)
E2	proof	formal/prefix_bmc/sby.log	present	smtbmc yices BMC depth 30 FAIL on 9e309e22
E3	proof	formal/prefix_cover/sby.log	present	pre-fix cover: CSR write, mscratch write, CSR in halted WB, halt_wb with valid WB all reached at step 8; 'illegal CSR with csr_en in WB' unreachable (EX clears csr_en for illegal accesses)
E4	proof	formal/master_prove/sby.log	present	master: abc pdr PASS unbounded for P1 and P2
E5	proof	formal/master_bmc/sby.log	present	master: BMC depth 30 PASS
E6	proof	formal/master_cover/sby.log	present	master cover: same 4 covers reached at step 7 -> the halted-WB situation still exists, only the write is suppressed

Paths are relative to runs/e40x-198. The File column is a check made when this document was generated. It reports whether the cited file exists, not whether its contents were re-validated. 'not kept' means a repository rule deliberately excludes that artefact, usually a waveform dump; rerun the check to regenerate it. 'MISSING' means the file is absent and was expected to be there.

What each class of artefact establishes

- counterexample. A concrete input sequence produced by the engine. Shows the stated condition is reachable; it is not itself a proof.
- proof. Proof or engine run. Scope depends on the mode: prove is unbounded under the stated model, bmc is bounded to its depth, cover establishes reachability only.

Artefact(s) E1 are not retained under repository policy. The result recorded against them stands on the engine log cited alongside; the artefact itself must be regenerated by rerunning the check before it can be inspected.

7 Limitations and scope

- The fix is verified against the checks listed under Evidence only. This is not a regression sign-off and does not cover the wider test suite.
- This is an internal investigation record. It requires engineering review before it is used as a disposition. No report generator can supply missing evidence.
- Results apply to the configuration and core revision named in Status. A different configuration needs a separate run.
- Public demonstration runs on open cores are not client engagements and not production verification results.

8 Provenance and measurement

Run directory

runs/e40x-198

Title source

Title and configuration read back from reports/issues/cv32e40x_issue_198.pdf, the previously published report for this issue.

Inputs shown	title, body(links stripped), labels
Human record	PR #201 'Halting CSR writes in WB' (d2ec2200, 2021-08-26): instr_valid = ex_wb_pipe_i.instr_valid && !kill_wb && !halt_wb in cs_registers; closes #197 and #198
Assessed confidence	0.92. This is the investigator's own estimate. It is not a measurement and not a reliability figure.
Iterations	1

Generated by reporting/scripts/issue_document.py from runs/e40x-198/verdict.json. The generator typesets recorded data and checks that cited files exist. It does not run, rerun or validate any check, and it cannot supply missing evidence.